Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

# Отчёт по модулю 1

курса «Системы искусственного интеллекта»

**Выполнили:** Ясаков Артём
Сарваров Тимур

**Группа:** P3313

**Проверил:** Жданов Андрей Дмитриевич

Санкт-Петербург, 2026 г.

# Contents

**1 Аннотация** **2**

**2 Введение** **2**

**3 База знаний в Prolog (ЛР1, часть 1)** **2**

3.1 Домен и словарь . . . 2
3.2 Соответствие минимальным требованиям . . . 2
3.3 Структура файла . . . 3
3.4 Набор запросов возрастающей сложности . . . 4

**4 Онтология в Protégé (ЛР1, часть 2)** **6**

4.1 Перевод Prolog → OWL . . . 6
4.2 Кардинальности и datatype-ограничения . . . 7
4.3 Проверка ризонером HermiT . . . 7
4.4 Мини-матрица трассируемости Prolog  OWL . . . 7
4.5 Скриншоты Protégé . . . 8
4.6 Компетентностные вопросы (DL Query) . . . 13

**5 DSS на базе знаний (ЛР2)** **14**

5.1 Формат входа и конвейер . . . 14
5.2 Ключевые фрагменты кода . . . 14
5.3 Примеры сценариев . . . 15
5.4 Нештатные случаи . . . 15

**6 Тестирование и отладка** **17**

6.1 Тест-кейсы Prolog . . . 17
6.2 Тест-кейсы DSS . . . 17

**7 Оценка и интерпретация** **17**

7.1 Prolog vs OWL . . . 17
7.2 Идеи развития . . . 18

**8 Заключение** **18**

**Использование искусственного интеллекта** **18**

**Список литературы** **19**

# 1 Аннотация

В рамках модуля 1 выполнены три лабораторные работы. Сначала спроектирована и реализована база знаний на языке Prolog для домена «Британская королевская семья»: 30 персон, события брака и развода, рекурсивные родственные связи и 38 правил вывода. Затем эта база знаний переведена в онтологию OWL в среде Protégé: классы `Person`, `Event`, `LivingPerson`/`DeadPerson`, объектные свойства `parentOf`, `ancestorOf` (транзитивное), `siblingOf` (симметричное), аксиомы цепочек свойств и кардинальности. Согласованность онтологии проверена ризонером HermiT. На основе базы знаний Prolog разработано CLI-приложение на Python (`pyswip` + `pymorphy3`), выполняющее роль системы поддержки принятия решений с естественно-языковым интерфейсом на русском языке. Проведены тестирование и сравнение Prolog и OWL по выразительности и удобству расширения.

# 2 Введение

**Цели модуля.** Освоить логическое моделирование предметной области, декларативную парадигму программирования, принципы построения онтологий и получить практический опыт разработки систем поддержки принятия решений (DSS) на основе базы знаний.

**Почему БЗ и онтологии полезны.**

- **Повторное использование знаний:** одна и та же БЗ/онтология обслуживает разные приложения — запросы, ризонинг, DSS.
- **Проверяемый вывод:** правило или аксиома формально описывают логику, результат воспроизводим и объясним.
- **Прозрачность:** в отличие от «чёрного ящика» ML, правила читаются человеком и легко правятся.

# 3 База знаний в Prolog (ЛР1, часть 1)

## 3.1 Домен и словарь

Домен — Британская королевская семья. Сущности: персоны (30 человек), события (10 браков, 3 развода). Отношения: пол, рождение/смерть, брак/развод, родитель-ребёнок. Производные отношения: мать/отец, сын/дочь, брат/сестра, дед/бабка, внук/внучка, дядя/тётя, племянник/племянница, двоюродные, предок/потомок, текущий/бывший супруг, вдовец, живой/умерший, старше/младше.

## 3.2 Соответствие минимальным требованиям

- **Унарные факты:** 60+ — `person/1` (30), `male/1` (14), `female/1` (16).

- **Бинарные и более:** 70+ — `birth/2`, `death/2`, `parent/2`, `marriage/3`, `divorce/3`.
- **Правил: 38**.

## 3.3 Структура файла

Файл `british_royal_family.pl` разбит на разделы с комментариями: персоны, пол, рождение, смерть, браки, разводы, родительские связи, правила (пронумерованы 1–38).

```
person(elizabeth_ii).
male(philip).
female(elizabeth_ii).
birth(elizabeth_ii, 1926).
death(elizabeth_ii, 2022).
marriage(elizabeth_ii, philip, 1947).
divorce(charles_iii, diana, 1996).
parent(elizabeth_ii, charles_iii).

% 6.
sibling(X, Y) :-
    parent(P1, X), parent(P1, Y),
    parent(P2, X), parent(P2, Y),
    P1 @< P2, X \= Y.

% 18.
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).

% 29.
alive(X) :- person(X), \+ death(X, _).

% 31.
current_spouse(X, Y) :-
    spouse(X, Y),
    \+ ex_spouse(X, Y),
    alive(X), alive(Y).
```

Listing 1: Фрагмент базы знаний

Полная БЗ: https://swish.swi-prolog.org/p/hZwWZIlp.pl.

### 3.4 Набор запросов возрастающей сложности

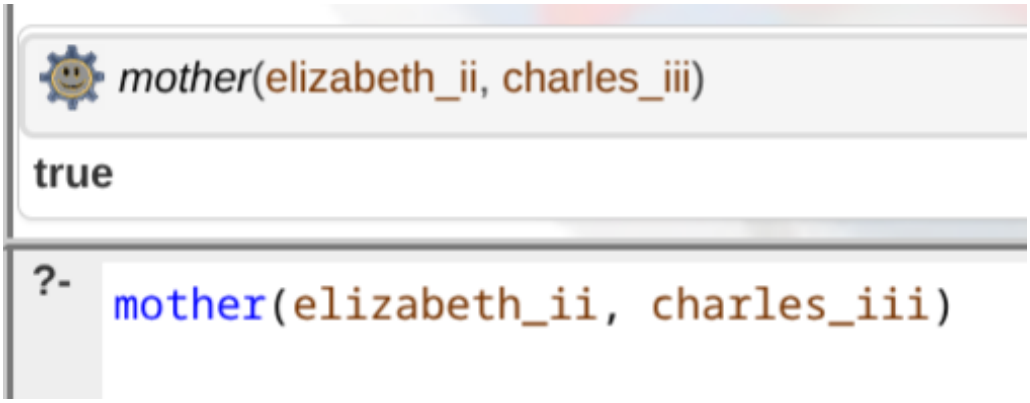



Figure 1: Запрос `?- mother(elizabeth_ii, charles_iii).` → `true`

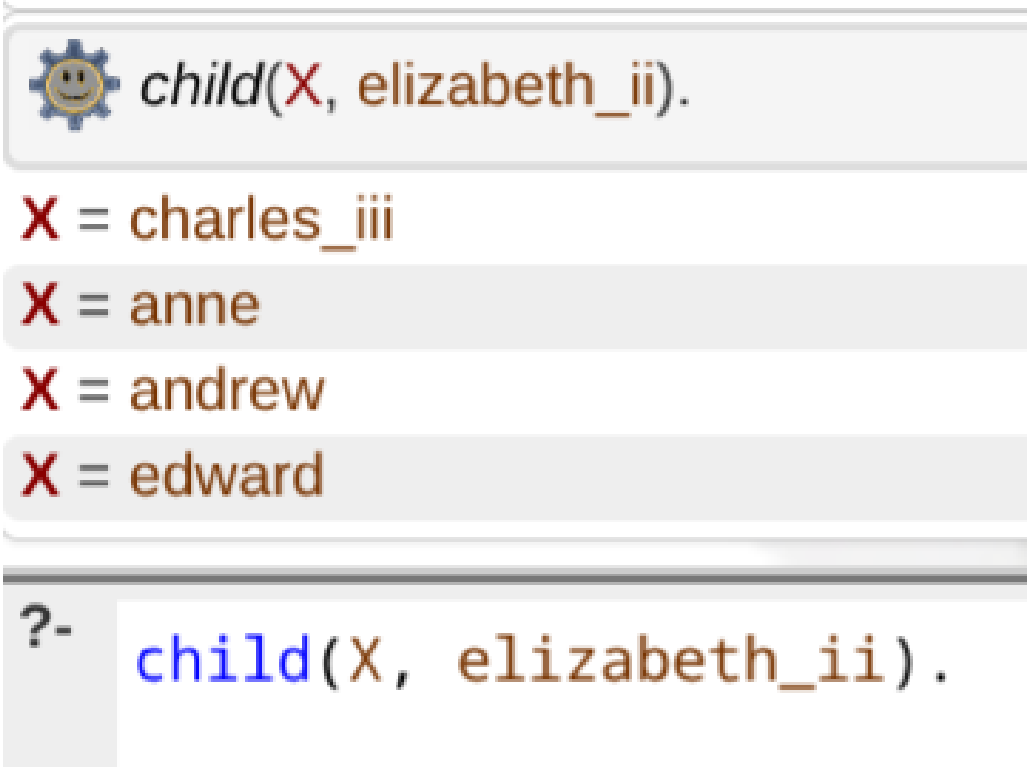



Figure 2: Запрос `?- child(X, elizabeth_ii).`

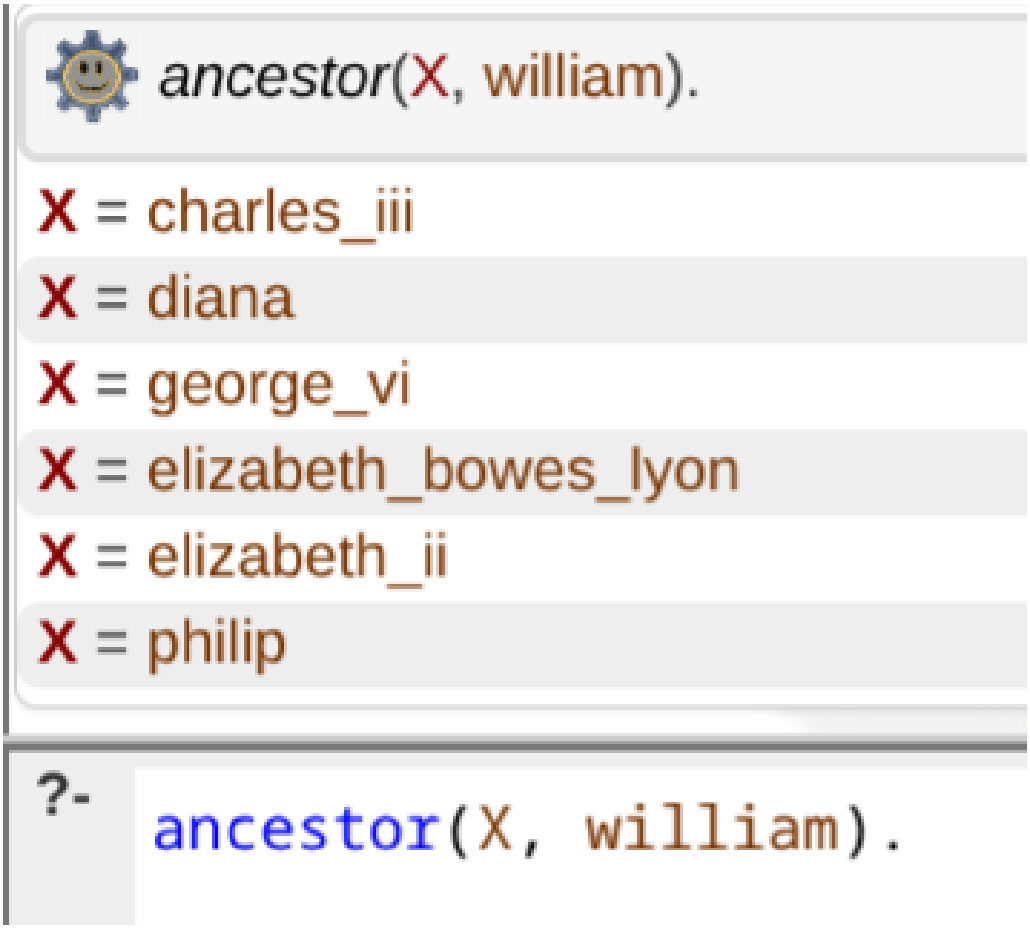



Figure 3: Запрос `?- ancestor(X, william).`




Figure 4: Запрос `?- childless(X), alive(X).`

# 4 Онтология в Protégé (ЛР1, часть 2)

## 4.1 Перевод Prolog → OWL

- **Классы:** `Person`, `Male`, `Female`, `LivingPerson`, `DeadPerson`, `MarriedPerson`, `SinglePerson`, `DivorcedPerson`, `Event`, `MarriageEvent`, `DivorceEvent`. Иерархия: `Person` → `Male`/`Female`, `Person` → `LivingPerson`/`DeadPerson`; `Event` → `MarriageEvent`/`DivorceEvent`.
- **Disjoint:** `Male` ⊓ `Female` ⊑ ⊥, `LivingPerson` ⊓ `DeadPerson` ⊑ ⊥.
- **ObjectProperty** с доменом/диапазоном: `parentOf` (Person → Person), `ancestorOf` (*Transitive*), `siblingOf` (*Symmetric*), `everSpouseOf`, `currentSpouseOf`, `exSpouseOf`,

grandparentOf.

- **DataProperty:** birthYear, deathYear, eventYear (xsd:integer).
- **Аксиома цепочки:** parentOf ∘ parentOf ⊑ grandparentOf.
- **Индивиды:** 30 персон, 13 событий.

## 4.2 Кардинальности и datatype-ограничения

На ключевых отношениях заданы кардинальности:

- Person ⊑ = 1 birthYear.xsd:integer — у каждой персоны ровно один год рождения.
- Person ⊑ ≤ 1 deathYear.xsd:integer — не более одной даты смерти.
- MarriageEvent ⊑ = 2 hasParticipant.Person — в браке ровно два участника.

Datatype-ограничения на значения:

- birthYear ≥ 1800 — отсекает заведомо некорректные значения.
- deathYear ≥ birthYear — проверяется через SWRL-правило или facet.

## 4.3 Проверка ризонером HermiT

После запуска ризонера (Reasoner → HermiT → Start reasoner) онтология проверена на согласованность: unsatisfiable-классы отсутствуют, событий логических противоречий не обнаружено. Disjoint-пары Male/Female и LivingPerson/DeadPerson работают корректно; ни один индивид не попал в оба класса одновременно.

## 4.4 Мини-матрица трассируемости Prolog OWL

| Prolog | OWL элемент | Комментарий |
|---|---|---|
| person/1 | :Person (класс) | Индивиды — экземпляры класса |
| male/1, female/1 | :Male, :Female | Disjoint-подклассы |
| parent/2 | :parentOf (ObjectProperty) | Домен/диапазон Person |
| ancestor/2 | :ancestorOf (Transitive) | Рекурсия → транзитивность |
| sibling/2 | :siblingOf (Symmetric) | Симметричность явная |
| alive/1 | :LivingPerson (класс) | Disjoint с DeadPerson |
| marriage/3 | :MarriageEvent + hasParticipant | Событие как индивид |

### 4.5 Скриншоты Protégé

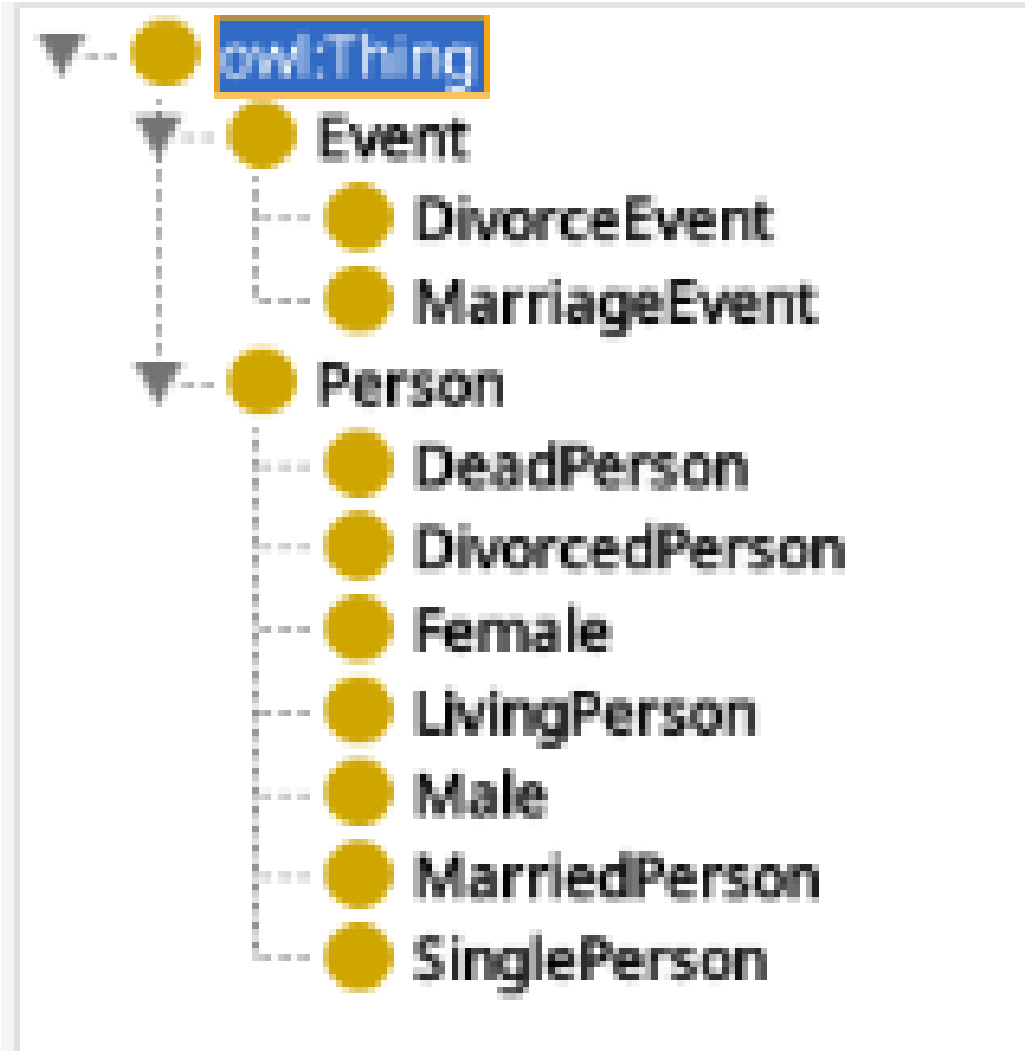



Figure 5: Иерархия классов онтологии

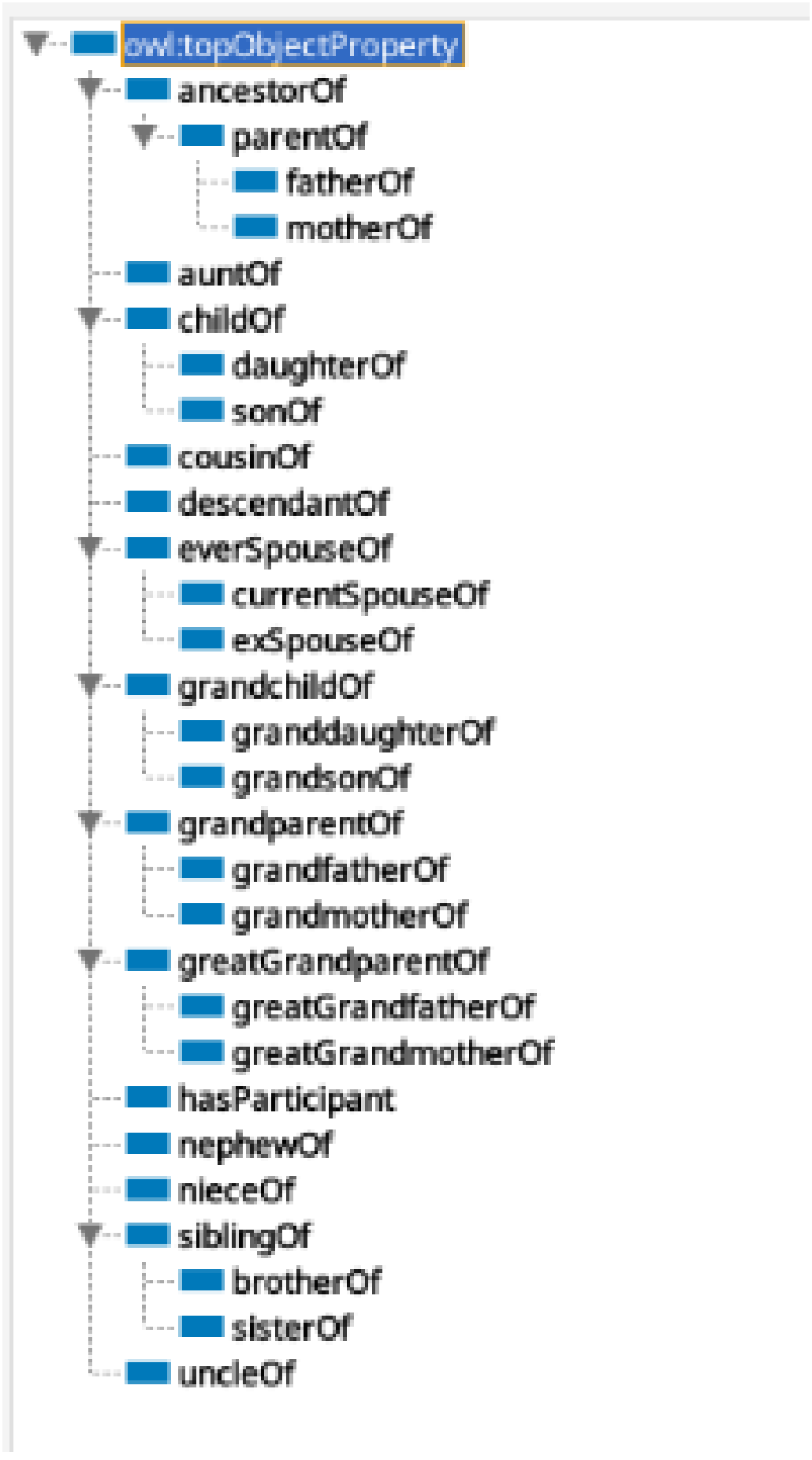



Figure 6: Иерархия объектных свойств

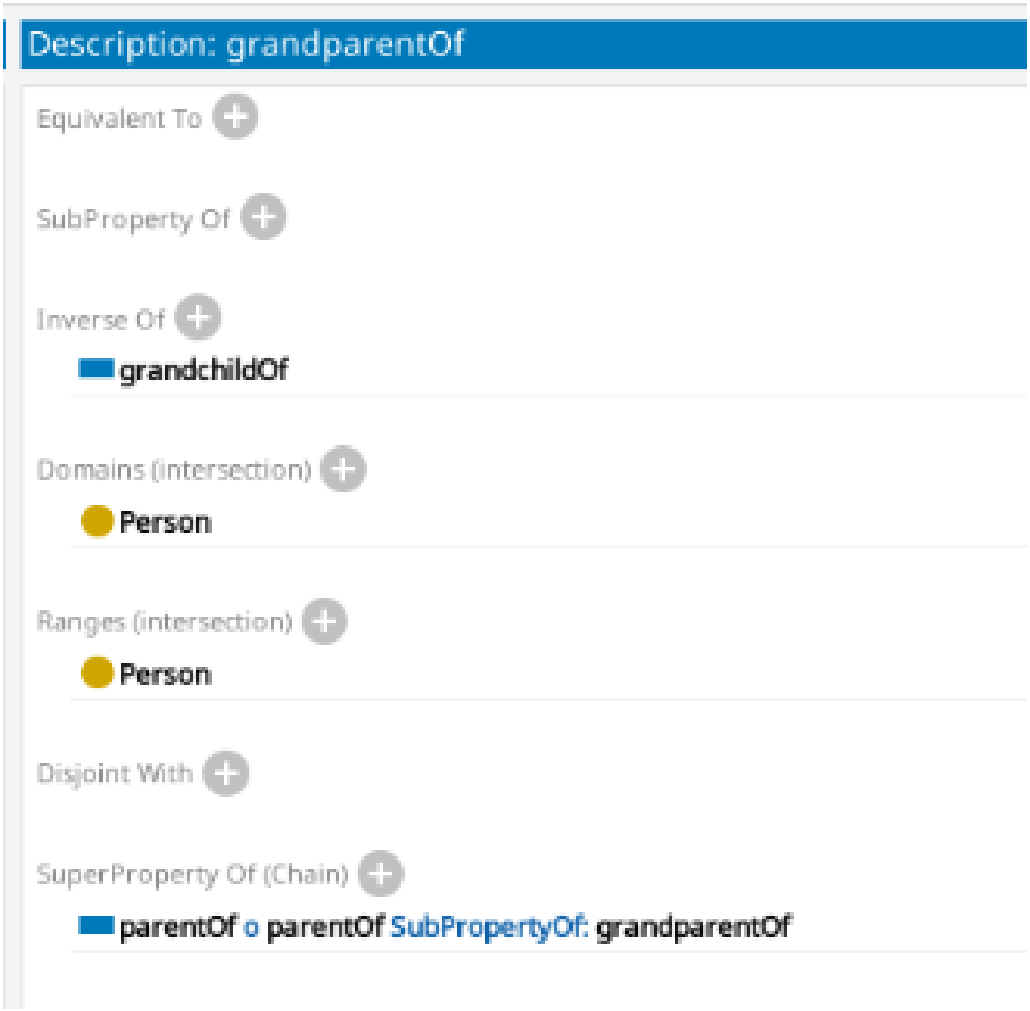



Figure 7: Настройка propertyChainAxiom для `grandparentOf`

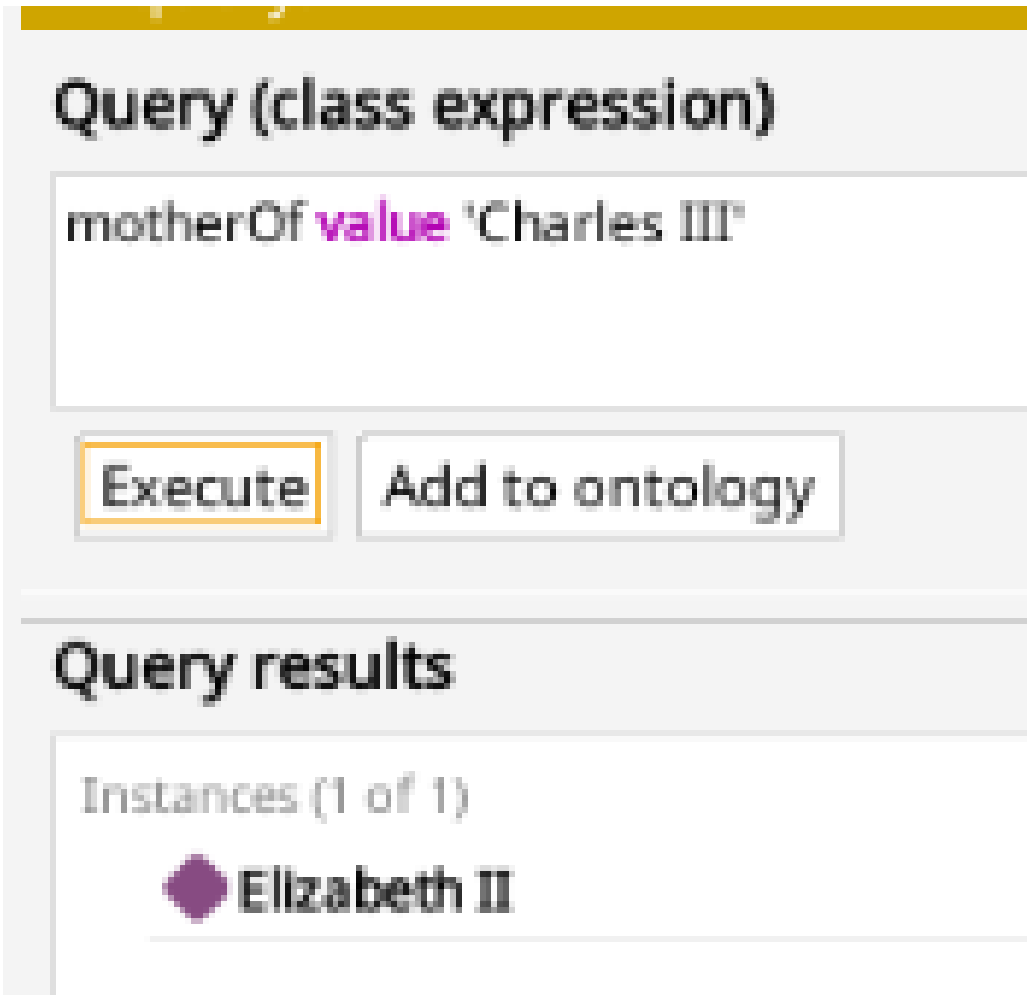



Figure 8: DL Query: `motherOf value charles_iii`

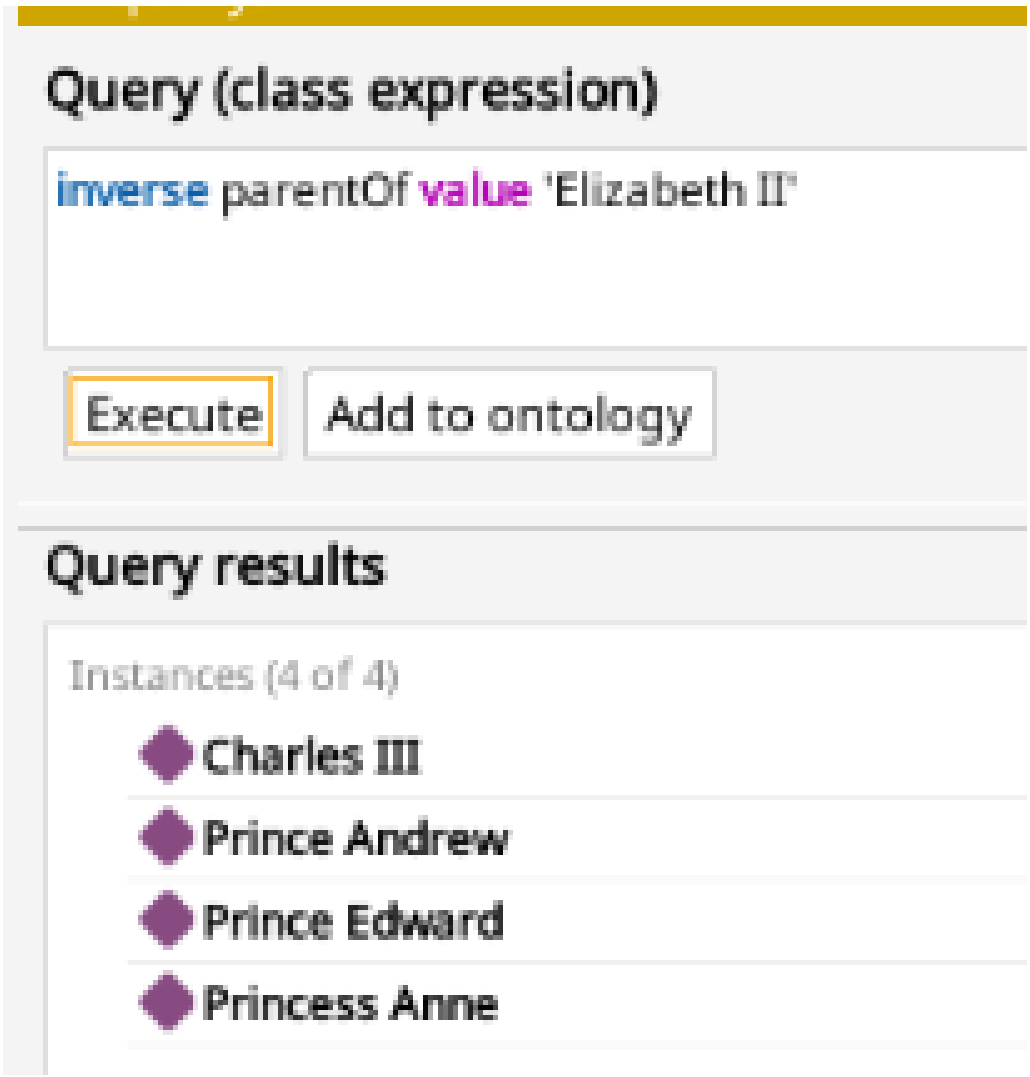



Figure 9: DL Query: `inverse parentOf value elizabeth_ii`

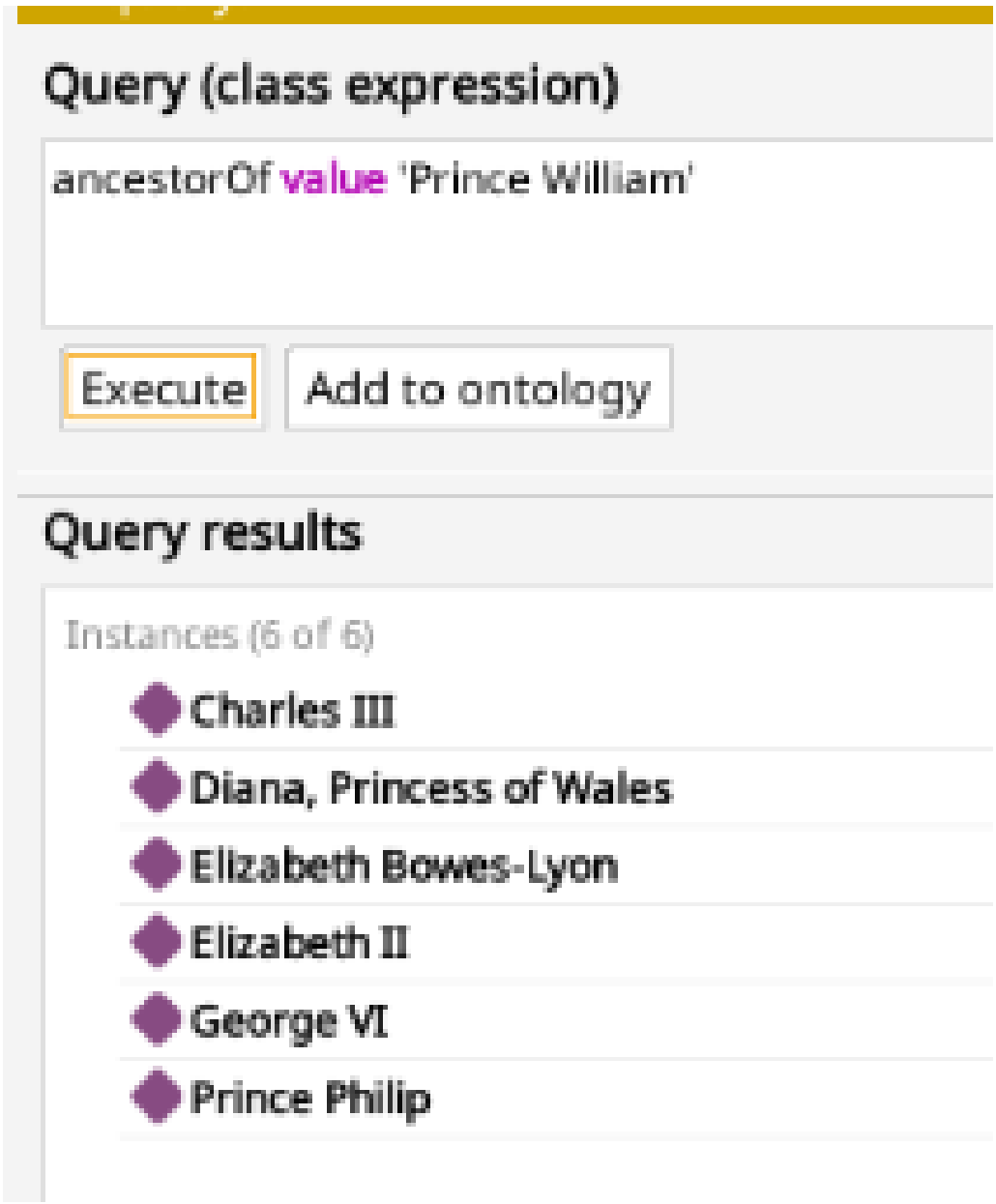



Figure 10: DL Query: `ancestorOf value william`

## 4.6 Компетентностные вопросы (DL Query)

| DL Query | Ожидаемый ответ |
|---|---|
| `motherOf value charles_iii` | `elizabeth_ii` |
| `inverse parentOf value elizabeth_ii` | 4 ребёнка |
| `ancestorOf value william` | 6 предков |
| `LivingPerson and (everSpouseOf some DeadPerson)` | `elizabeth_ii` |
| `LivingPerson and (parentOf max 0 Person)` | пусто (OWA) |

# 5 DSS на базе знаний (ЛР2)

## 5.1 Формат входа и конвейер

CLI-приложение `recommender.py` принимает русскоязычный запрос фиксированного вида, например: `«найди детей Елизаветы II»` или `«кем является Чарльз III для Елизаветы II»`.

Конвейер обработки:

1. **Парсинг** строки через regex → слоты `(action, arg1, arg2)`.
2. **Нормализация** падежей через `pymorphy3` (`елизаветы` → `елизавета`).
3. **Поиск персоны** — пробуем сначала исходную строку, потом нормализованную.
4. **Формирование Prolog-запроса** через `pyswip`: `find_relatives/4` или `what_is/3`.
5. **Агрегация результата** — удаление дубликатов, вывод меток на русском.

## 5.2 Ключевые фрагменты кода

```
import re
from pyswip import Prolog
import pymorphy3

prolog = Prolog()
prolog.consult("royal_family.pl")
morph = pymorphy3.MorphAnalyzer()

def normalize_word(word):
    w = word.lower().strip()
    if not re.search(r'[  -    ]', w):
        return w
    return morph.parse(w)[0].normal_form

def find_person(name_words):
    raw  = ' '.join(w.lower() for w in name_words)
    norm = ' '.join(normalize_word(w) for w in name_words)
    for candidate in (raw, norm):
        q = f"name_form('{candidate}', Id)"
        results = list(prolog.query(q))
        if results:
            return results[0]['Id']
    return None
```

```
def parse_query(text):
    low = text.strip().lower()
    m = re.match(r'^(?:           \s+)?      \s+                \s
       +(.+?)\s+      \s+(.+)$', low)
    if m:
        return ('what', m.group(1).split(), m.group(2).split())
    m = re.match(r'^(?:           \s+)?(\S+)\s+(.+)$', low)
    if m:
        return ('rel', m.group(1), m.group(2).split())
    return (None, None, None)
```

Listing 2: Фрагменты `recommender.py`

## 5.3 Примеры сценариев

- `найди детей Елизаветы II` → `child(X, elizabeth_ii)` → 4 ребёнка.
- `найди братьев Уильяма` → `brother(X, william)` → `harry`.
- `найди предков Елизаветы II` → `ancestor(X, elizabeth_ii)` → 2 предка.
- `найди супругов Елизаветы II` → `spouse(elizabeth_ii, X)` → `philip`.
- `кем является Чарльз III для Елизаветы II` → `what_is(charles_iii, elizabeth_ii, Rel)` → «сын».

## 5.4 Нештатные случаи

- Неизвестное отношение → сообщение со списком доступных.
- Персона не найдена → сообщение «Персона … не найдена в базе».
- Пустой результат → «У … нет таких родственников».
- Команды `выход`/`exit` и `помощь`/`help` обрабатываются до парсинга.

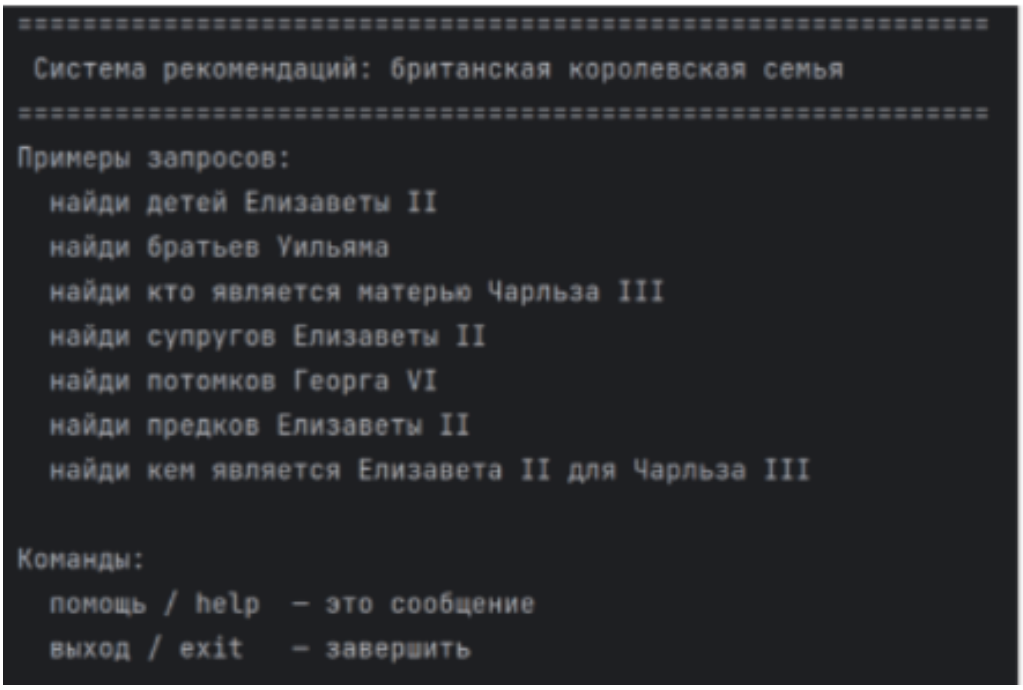



Figure 11: Справка CLI-приложения

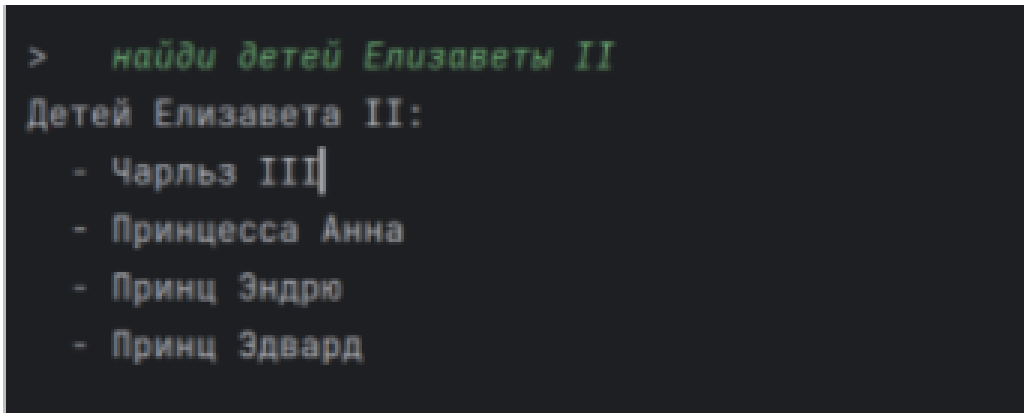



Figure 12: Запрос «найди детей Елизаветы II»

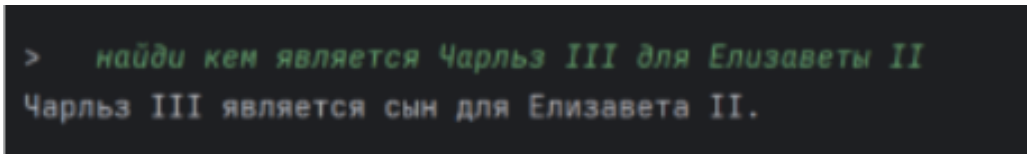



Figure 13: Обратный запрос «кем является Чарльз III для Елизаветы II»

# 6 Тестирование и отладка

## 6.1 Тест-кейсы Prolog

| Запрос | Ожидание |
|---|---|
| `mother(elizabeth_ii, charles_iii).` | `true` |
| `child(X, elizabeth_ii).` | 4 решения |
| `ancestor(X, william).` | 6 решений |
| `widowed(elizabeth_ii).` | `true` |
| `current_spouse(charles_iii, camilla).` | `true` |
| `older(charles_iii, william).` | `true` |

## 6.2 Тест-кейсы DSS

- **Happy path:** «найди детей Елизаветы II» → 4 персоны.
- **Негативный:** «найди детей Обамы» → персона не найдена.
- **Негативный:** «найди кого-нибудь» → неизвестное отношение.
- **Служебный:** `help`, `exit`.

# 7 Оценка и интерпретация

## 7.1 Prolog vs OWL

| Параметр | Prolog | OWL (Protégé) |
|---|---|---|
| Мир | Замкнутый (CWA) | Открытый (OWA) |
| Вывод | Backtracking | Reasoning |
| Время/арифм. | Высокая (сравнение дат) | Через SWRL |
| Правила | `:-` | propertyChain + SWRL |
| Расширение | Быстрое (новый факт) | Через классы/индивиды |

Prolog удобнее для арифметики, сравнений и рекурсии (например, `older/2`). OWL удобнее для расширения таксономии и явной проверки согласованности.

### 7.2 Идеи развития

- Добавить SWRL-правила в онтологию.
- Подключить `owlready2` для запросов к OWL из CLI.
- Расширить диалог до многошагового сценария DSS.

## 8 Заключение

Реализованы три компонента: база знаний Prolog (30 персон, 38 правил), онтология OWL в Protégé (классы, свойства, аксиомы цепочек, кардинальности, 30 индивидов) и CLI-приложение на Python с естественно-языковым интерфейсом. Согласованность онтологии подтверждена ризонером HermiT. Все требования модуля выполнены. Ограничение — отсутствие SWRL в онтологии и одноразовость диалога; оба пункта вынесены в план развития. Подход с прозрачными правилами обеспечивает воспроизводимость вывода и удобство расширения.

## Использование искусственного интеллекта

При подготовке отчёта использовались инструменты искусственного интеллекта в качестве вспомогательного средства:

- для разбора формулировок задания и уточнения требований методического пособия;
- для проверки логики реализации базы знаний Prolog, онтологии OWL и CLI-приложения;
- для пояснения отдельных этапов обработки данных и нормализации естественно-языковых запросов;
- для редакторской правки формулировок и улучшения оформления отчёта.

Весь код, структура базы знаний, онтология и итоговые решения разработаны авторами самостоятельно. ИИ применялся исключительно как консультационный и редакторский инструмент, вся ответственность за корректность результатов лежит на авторах.

# Список литературы

1. Бессмертный И.А., Авдюшина А.Е., Кугаевских А.В., Королева Ю.А., Рущенко Н.Г. Системы искусственного интеллекта: учебно-методическое пособие. — СПб.: Университет ИТМО, 2024. — 81 с.

2. Бессмертный И.А. Искусственный интеллект. — СПб.: Университет ИТМО, 2010. — 133 с. URL: https://books.ifmo.ru/file/pdf/658.pdf

3. Кугаевских А.В., Муромцев Д.И., Кирсанова О.В. Классические методы машинного обучения. — СПб.: Университет ИТМО, 2022. — 52 с. URL: https://books.ifmo.ru/file/pdf/3075.pdf

4. Horridge M. A Practical Guide To Building OWL Ontologies Using Protégé 4 and CO-ODE Tools. — Manchester: University of Manchester, 2011. — 107 p.

5. SWI-Prolog Reference Manual. URL: https://www.swi-prolog.org/pldoc/

6. PySwip Documentation. URL: https://github.com/yuce/pyswip

7. Pymorphy3 — морфологический анализатор русского языка. URL: https://pypi.org/project/pymorphy3/

8. Protégé. URL: https://protege.stanford.edu/

9. W3C OWL 2 Web Ontology Language Primer (Second Edition). URL: https://www.w3.org/TR/owl2-primer/